DAY 2 — Working with NumPy & Numerical Computing

Day 2 focused entirely on **NumPy**, the backbone of numerical computing in Python. NumPy makes mathematical operations extremely fast, especially when dealing with large datasets like stock prices, financial records, or sensor data.

I started by understanding **NumPy Arrays**, which are more efficient than Python lists because they store data in a continuous memory block. This allows faster computation and better performance in data-heavy environments.

which makes computations significantly faster. This is especially useful when working with large datasets like stock market prices.

🔹 **Key Concepts Learned:**

- Creating arrays using
  np.array(), np.zeros(), np.ones(), np.arange(), np.linspace()
- Array indexing and slicing
- Changing shapes and dimensions using
  .reshape(), .ravel(), .flatten()
- Mathematical operations like
  addition, multiplication, square root, exponent, dot product
- Statistical functions:
  mean, median, variance, standard deviation

🔹 **Broadcasting**

One of the most powerful features of NumPy is **broadcasting**, which allows operations between arrays of different shapes without manual resizing.
Example: adding a number to an entire array automatically applies to every element.

🔹 **Why NumPy Matters**

NumPy is used internally by many advanced libraries:

- Pandas (for tables)
- Scikit-Learn (for machine learning)
- TensorFlow & PyTorch (deep learning)
- Prophet (time series forecasting)

Learning NumPy made me comfortable handling numerical datasets and performing analytical computations.

⭐ **Key Takeaways from Day 2:**

- NumPy arrays are faster and more memory-efficient
- Broadcasting simplifies calculations

- Most AI/ML frameworks depend on NumPy
- Strong NumPy skills make data analysis much easier